BVS1 – Betriebssysteme

Lecture 02 – From Source to Process · Wintersemester 2026/2027

Stavros Kousidis

TH Köln · ICCT

From Source to Process

We Already Know the End

Last time we observed:

```
Hello BVS1
```

on an RV64 target with **no guest operating system**.

Today:

> *How did our source code become something that this target could execute?*

And this time we will unfold the build ourselves.

Prediction — What Does QEMU Receive?

Which of these do we give to QEMU to start our RV64 target?

1. `main.c`
2. `start.S`
3. an object file such as `main.o`
4. the linked ELF
5. the Makefile

Commit to one answer before we inspect anything.

Step 1 — Compile the C Source

Create a separate manual-build directory:

```
mkdir -p build/manual
```

Then compile **without linking**:

```
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -msmall-data-limit=0 -c main.c -o build/manual/main.o
```

The important option today is:

```
-c  → compile/assemble, but do not link
```

Predict Before We Inspect

We have compiled `main.c`, but we have **not linked**.

Before running `file`, predict:

1. **Has the target architecture already been decided?**
2. **Which architecture should `file` report?**
3. **Is `main.o` already the final executable?**

Then test your prediction:

```
file build/manual/main.o
```

What Did `file` Tell Us?

Typical output contains:

```
ELF 64-bit ... relocatable ... RISC-V ...
               |                  |
          not final yet       target ISA
```

So:

- the target architecture is already decided;
- `main.o` is not the final linked program;
- the source was C, but the object file is already target-specific: RISC-V.

Step 2 — Assemble the Startup Code

Now translate the startup assembly:

```
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -c start.S -o build/manual/start.o
```

Inspect both object files:

```
riscv64-unknown-elf-nm build/manual/start.o
riscv64-unknown-elf-nm build/manual/main.o
```

Look only for:

- symbols this object file **provides**;
- symbols it **references but cannot yet resolve**.

Why Can’t We Run the Object Files Yet?

We can already see:

```
start.o defines _start
start.o needs   main
start.o needs   _stack_top

main.o  defines main
```

So one question remains:

> *Who provides **`_stack_top`**, and who decides where all of this lives in target memory?*

Back to the Stored-Program Idea

Lecture 1

```
instructions + data
        ↓
      memory
        ↕
       CPU
```

Now

```
instructions
constant data
writable data
stack
```

Where should these pieces live in our RV64 memory?

Our RV64 Program in Memory

One memory — different roles for different bytes.

`sp` = stack pointer

```
RV64 memory from 0x80000000


+-----------------------------------------------------------+
| .text    _start, main, ...      | instructions          |
| .rodata  "Hello BVS1\n"         | constant data         |
| .data                           | initialized data      |
| .bss                            | zero-initialized data |
| C stack  (top: _stack_top)      | runtime data          |
+-----------------------------------------------------------+
```

The Linker Script Makes the Layout Concrete

`linker.ld` is **course-provided**; QEMU provides the machine, not the script.

```
ENTRY(_start)

MEMORY {
    RAM (rwx) : ORIGIN = 0x80000000, LENGTH = 128M
}

. = ALIGN(16);
. += 16K;
_stack_top = .;
```

`.` = current output address. After reserving 16 KiB, that address is named `_stack_top`.

Step 3 — Link the Program

```
riscv64-unknown-elf-gcc -nostdlib -nostartfiles -no-pie \
  -T linker.ld build/manual/start.o build/manual/main.o \
  -o build/manual/hello-bvs1.elf
```



The linker now:

- combines the objects,
- resolves symbols,
- establishes the final memory layout.

Inspect the Final ELF

```
file build/manual/hello-bvs1.elf
riscv64-unknown-elf-nm build/manual/hello-bvs1.elf
```

Compare with `start.o`.

Questions:

- Is `main` still unresolved?
- Does `_stack_top` now have an address?
- Which architecture is the executable for?

Where Does Execution Begin?

The linker script told us:

```
ENTRY(_start)
```

And `nm` (“name list”) shows that `_start` now has an address.

So when the RV64 machine starts our program:

> *execution begins at* ***`_start`***

Where did `_start` come from?

Our First RISC-V Assembly

From `start.S`:

```
_start:
    la   sp, _stack_top
    call main
1:
    j    1b
```

Read it as a story:

- set `sp` to `_stack_top`;
- call `main`;
- if `main` returns, stay here forever.

Step 4 — Run It Without Make

```
qemu-system-riscv64 -machine virt -m 128M -smp 1 -nographic \
  -bios none -kernel build/manual/hello-bvs1.elf
```

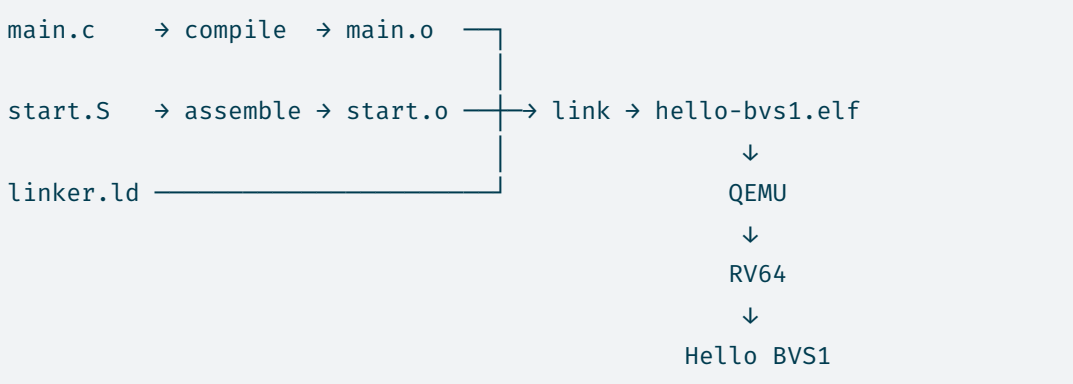
Expected:

```
Hello BVS1
```

Exit QEMU with **Ctrl-a**, release, then **x**.

We have reproduced the experiment without `make`.

Reconstruct the Complete Path



Which arrows happen on the development host? Which on the target?

Change, Rebuild, Observe

Our First Source-Code Change

Find:

```
void main(void)
{
    uart_puts("Hello BVS1\n");
    for (;;) {}
}
```

Change **only the string**, for example:

```
uart_puts("Hello from <your name>\n");
```

Do not rebuild yet.

Prediction — What Must Change?

We changed only `main.c`.

Which artefacts must change before the new text can appear?

- `main.o`
- `start.o`
- `linker.ld`
- `hello-bvs1.elf`
- QEMU itself

Explain your prediction before rebuilding.

Rebuild Only What Is Necessary

Recompile `main.c`:

```
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -msmall-data-limit=0 -c main.c -o build/manual/main.o
```

Then link again.

Do **not** rebuild `start.o`.

Run the new ELF and compare prediction with observation.

Who Does What?

Make — automates the workflow; it does not compile or link by itself.

- decides whether the target is out of date;
- runs the recipe.

GCC driver

- compiles `main.c`, assembles `start.S`, and invokes the linker.

QEMU

- provides the emulated RV64 machine and loads the ELF.

Toward a Process

One Execution Owns the CPU

Our current path is simple:

```
_start
  ↓
main
  ↓
print
  ↓
for (;;) {}
```

The RV64 CPU keeps executing this one flow forever.

Who can decide to run something else?

The Requirement Changes

Suppose the software we are building should become an operating system.

Then it must be able to:

```
run some execution
       ↓
regain control
       ↓
continue that execution later
```

The execution must therefore become something the OS can **manage**.

Give the Managed Execution a Name

For our first operating-system model:

> *A **process** is an execution that the operating system can manage.*

For **Checkpoint 1 — I have a process.**, "manage" means only:

```
create → run → yield → resume → exit
```

Nothing more yet.

Our First Process Lifecycle



A yielded process may run again.

A finished process must not.

Yield Is Not Exit

yield

execution is **not finished**

the OS regains control

the process must be able to continue later

exit

execution is **finished**

the OS regains control

the process must never continue

Same control transfer. Different meaning.

Resume Is Not Restart

Imagine a process executes:

```
uart_puts("A\n");
process_yield();
uart_puts("B\n");
process_exit();
```

`process_yield()` and `process_exit()` are operations we still have to implement.

After `yield`, the OS has control.

When the process runs again:

> *Should it print **A** or **B** next?*

The Problem We Have Not Solved

We now want:

```
process runs
     ↓
yield
     ↓
OS regains control
     ↓
resume
     ↓
continue exactly where it stopped
```

What must survive the pause so that this is possible?

Before You Leave

Today we followed one straight path:

```
source → objects → linked ELF → memory → _start → main
```

From there:

```
executing program → OS-managed execution → process
```

A process must be able to:

```
create → run → yield → resume → exit
```

Next:

What machine state makes resume possible?